ACTIVIDAD ANÁLISIS DE BIG DATA – INFORME TÉCNICO

Análisis de la oferta de servicios de salud de la red pública de Bogotá D. C. mediante Apache Hadoop y el paradigma MapReduce

PRESENTADO POR:
DANIEL FELIPE LEÓN HIGUERA

DOCENTE: EDGAR MAURICIO LÓPEZ ROJAS
ESPECIALIZACIÓN EN ANALÍTICA DE DATOS
UNIVERSIDAD DE CUNDINAMARCA
CHÍA, NOVIEMBRE DE 2026

REPOSITORIO DE GITHUB
https://github.com/Daniellеon172025/repo-mapreduce-bigdata

TABLA DE CONTENIDO

1. Introducción.....3
1.1 Objetivos.....3
1.2 Alcance.....4
2. Marco Conceptual.....4
2.1 El Paradigma MapReduce.....4
2.2 Componentes del Procesamiento.....5
2.3 Apache Hadoop en el Ecosistema Big Data.....5
2.4 Procesamiento, Análisis y Visualización.....6
2.5 Análisis de Datos y Toma de Decisiones.....6
3. Metodología.....6
3.1 Datos de Origen.....6
3.2 Entorno de Ejecución.....7
3.3 Diseño de los Jobs MapReduce.....10
4. Implementación.....12
4.1 Estructura del Repositorio.....12
4.2 Código Fuente del Job.....13
4.3 Script de Compilación y Ejecución.....14
4.4 Ejecución del Procesamiento.....15
5. Resultados.....20
5.1 Contadores de Ejecución.....20
5.2 Sedes por Localidad.....21
5.3 Sedes por Subred.....23
5.4 Distribución por Zona y Verificación del Nivel.....24
5.5 Validación Cruzada con Apache Spark.....25
6. Discusión.....26
6.1 Interpretación para la Toma de Decisiones.....26
6.2 Limitaciones.....26
7. Conclusiones.....27
8. Repositorio de GitHub.....27
9. Referencias.....28
Anexo A. Código Fuente de ConteoSedes.java.....29
Anexo B. Script ejecutar_mapreduce.sh.....31

1. Introducción

MapReduce es un modelo de programación que usa procesamiento paralelo: divide una tarea grande en muchas tareas pequeñas que se ejecutan al mismo tiempo y luego integra sus resultados (Holdsworth & Kosinski, s.f.). Es uno de los componentes centrales de Apache Hadoop, junto con el sistema de archivos distribuido de Hadoop (HDFS, por las siglas en inglés de Hadoop Distributed File System) y el gestor de recursos YARN (Yet Another Resource Negotiator).

Este informe documenta el análisis, mediante MapReduce, de los datos que se cargaron y limpiaron en actividades anteriores: las sedes de la red pública de prestadores de servicios de salud de Bogotá D. C. registradas en el Registro Especial de Prestadores de Servicios de Salud (REPS), con 103 sedes. El trabajo aplica los conceptos de la *Guía Big Data – Análisis* (Universidad de Cundinamarca, s.f.) y toma como criterio técnico el tutorial oficial de MapReduce de Apache Hadoop (Apache Software Foundation, 2023) y la descripción del paradigma de Holdsworth y Kosinski (s.f.). Se desarrolló un job en Java que responde cuatro preguntas de planeación en salud: cómo se distribuye la oferta entre localidades, qué proporción de las sedes atiende las 24 horas, qué subred concentra más sedes y cuánta oferta es rural. El código y los resultados están en un repositorio de GitHub (sección 8).

1.1 Objetivos

General: analizar con MapReduce, sobre un clúster Apache Hadoop, los datos procesados del REPS, interpretando los resultados para apoyar decisiones de planeación en salud.

Específicos:

- Explicar el funcionamiento de MapReduce (map, combine, shuffle/sort, reduce) y el papel de Hadoop en un ecosistema Big Data.

- Implementar y ejecutar un job MapReduce en Java sobre datos almacenados en HDFS, documentando scripts, comandos y configuración.
- Interpretar técnicamente los contadores de ejecución y los resultados analíticos obtenidos.
- Validar los resultados contrastándolos con los obtenidos con Apache Spark sobre los mismos datos.
- Documentar el procedimiento bajo normas APA 7.ª edición y publicarlo en un repositorio de GitHub.

1.2 Alcance

El análisis se realiza sobre un único archivo CSV de 57 837 bytes, en un clúster HDFS de un nodo desplegado con Docker. El ejercicio es formativo y cubre el diseño y la ejecución de un flujo MapReduce completo. No mide rendimiento a gran escala ni usa YARN con varios nodos.

2. Marco Conceptual

2.1 El Paradigma MapReduce

MapReduce procesa los datos como pares clave-valor. El framework recibe un conjunto de pares de entrada y produce otro conjunto de pares de salida, que pueden ser de tipos distintos; las clases de clave y valor deben ser serializables y las claves, además, comparables para poder ordenarse (Apache Software Foundation, 2023). El flujo lógico de un job es: entrada, map, combine, reduce y salida. Según Holdsworth y Kosinski (s.f.), los datos de entrada, normalmente desde HDFS, se dividen en bloques que se reparten entre los mapeadores. Cada mapeador los convierte en pares clave-valor. En la etapa de shuffle y sort, el framework ordena los resultados y envía todos los valores con la misma clave a un mismo reductor. Los reductores los agregan para producir la salida.

Tabla 1

Fases de MapReduce y su implementación en este trabajo

Fase	Descripción conceptual	Implementación en el job
Entrada y división	El InputFormat valida la entrada, la divide en InputSplit lógicos (uno por mapper) y entrega los registros mediante un RecordReader.	TextInputFormat (por defecto): un solo split, porque el archivo (57 837 bytes) es menor que un bloque HDFS. Cada registro es (desplazamiento, línea).
Map	Transforma cada registro de entrada en cero o más pares intermedios.	SedeMapper: analiza la línea y emite (clave, "1,e"), con e = 1 si la sede declara atención 24 h.
Combine	Agregación local de la salida de cada mapper, para reducir los datos que viajan al reducer.	SumaReducer reutilizado como Combiner (la suma es asociativa y conmutativa).
Partición	El Partitioner decide a qué reducer va cada clave.	HashPartitioner (por defecto) con un solo reducer.
Shuffle y sort	Cada reducer obtiene su partición de todos los mappers y agrupa los valores por clave.	Un reducer recibe las claves ordenadas y agrupadas.
Reduce	Agrega los valores asociados a cada clave.	SumaReducer suma ambos contadores por clave.
Salida	El OutputFormat escribe los pares finales.	TextOutputFormat: archivo part-r-00000 en HDFS, con formato clave, tabulador, valor.

Nota. Elaboración propia a partir de Apache Software Foundation (2023) y Holdsworth y Kosinski (s.f.).

2.2 Componentes del Procesamiento

Además de las clases anteriores, el tutorial de Hadoop describe los contadores (Counters), estadísticas globales que el framework o la propia aplicación actualizan desde las funciones map y reduce y que Hadoop agrega al finalizar el job (Apache Software Foundation, 2023). En una ejecución sobre YARN, el ResourceManager actúa como maestro, cada nodo worker ejecuta un NodeManager y cada aplicación tiene un MRAppMaster que planifica las tareas; si una tarea falla, el framework la vuelve a ejecutar. En este trabajo se emplean los contadores personalizados para auditar la calidad de la lectura (líneas leídas, válidas y descartadas).

2.3 Apache Hadoop en el Ecosistema Big Data

Apache Hadoop es un framework de código abierto, escrito en Java, para almacenar y procesar grandes volúmenes de datos en clústeres (Apache Hadoop, 2026; Holdsworth & Kosinski, s.f.). HDFS provee el almacenamiento distribuido y tolerante a fallos, YARN gestiona

los recursos y MapReduce aporta el modelo de procesamiento. Según Holdsworth y Kosinski (s.f.), entre sus ventajas están la escalabilidad a petabytes y el procesamiento paralelo cerca de los datos. Entre sus limitaciones, normalmente no retiene datos en memoria y la lógica iterativa obliga a encadenar varios jobs; muchas organizaciones están migrando a alternativas como Apache Spark. Por eso resulta útil contrastar ambos motores sobre los mismos datos, como se hace en la sección 5.5.

2.4 Procesamiento, Análisis y Visualización

En una arquitectura Big Data, el procesamiento prepara los datos (limpieza, normalización y transformación), el análisis los resume en indicadores y la visualización comunica esos indicadores a quienes toman decisiones. Aquí la secuencia es la misma: Spark limpió los datos en la actividad anterior, el job MapReduce los agrega por localidad, subred y zona, y matplotlib los grafica. Holdsworth y Kosinski (s.f.) incluyen entre los usos típicos de MapReduce la integración de datos, el análisis de registros y la tabulación, que es el tipo de agregación que se implementa aquí.

2.5 Análisis de Datos y Toma de Decisiones

Saber qué localidades concentran la oferta, cuáles no tienen atención permanente y qué subredes dependen menos de servicios 24 horas permite priorizar inversión y revisar horarios. Los resultados se interpretan en la sección 6.

3. Metodología

3.1 Datos de Origen

Se utilizó el conjunto de datos abiertos *Oferta de instituciones prestadoras de servicio de salud de la red pública de Bogotá D. C.*, publicado por la Secretaría Distrital de Salud en datos.gov.co (Secretaría Distrital de Salud de Bogotá, 2026). El archivo CSV (texto con valores separados por un delimitador) tiene 103 sedes y 52 columnas, usa punto y coma como separador, está codificado en ISO-8859-1 y trae como fecha de corte del REPS el 22 de septiembre de 2026.

En la actividad anterior el archivo se cargó en HDFS (/data/input) y se limpió con Spark, que generó una versión en Parquet (/data/output/reps_parquet). Para este análisis se usa el CSV original almacenado en HDFS, porque el MapReduce nativo de Hadoop lee texto plano mediante TextInputFormat. El resultado de Spark se usa después como referencia de validación.

Tabla 2

Campos del conjunto de datos utilizados por el job

Columna	Campo	Uso en el análisis
5	nombre_prestador	Subred integrada de servicios de salud (modo subred)
10	tipo_zona	Zona urbana o rural (modo zona)
15	Localidad	Código de la localidad (modo localidad)
16	Localidad_codigo	Nombre de la localidad (modo localidad)
32	nivel	Nivel de complejidad (modo nivel, verificación de calidad)
37 a 43	horario_lunes … horario_domingo	Detección de atención 24 horas

Nota. Elaboración propia a partir del esquema del archivo REPS. La numeración corresponde a la posición de la columna comenzando en 1.

En el archivo fuente los campos Localidad y Localidad_codigo vienen intercambiados: el primero contiene el código numérico y el segundo el nombre. El job los combina en una sola clave del tipo "8 - KENNEDY".

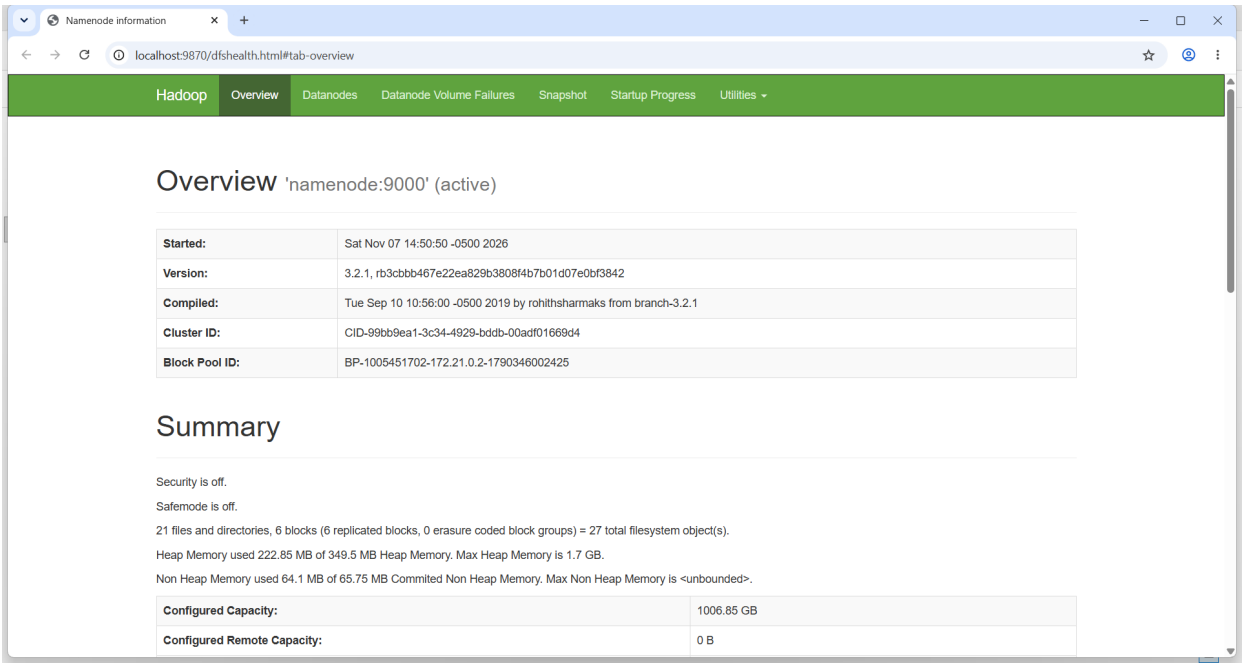
3.2 Entorno de Ejecución

El clúster corre sobre Windows 11 con Docker Desktop 4.74.0 (Docker Inc., 2026) y Docker Engine 29.4.3. Los contenedores namenode y datanode usan las imágenes bde2020/hadoop-namenode y hadoop-datanode (Hadoop 3.2.1, OpenJDK 1.8.0_232) y están definidos en el docker-compose.yml del proyecto Hadoop-spark-docker. El job se compila y se ejecuta dentro del contenedor namenode, con javac, hadoop classpath y hadoop jar. Como el docker-compose.yml no incluye un NodeManager de YARN, los jobs se ejecutan con el ejecutor local de Hadoop (LocalJobRunner), indicado con el parámetro -D mapreduce.framework.name=local. Los jobs leen y escriben en HDFS, pero sus tareas map y reduce corren en una sola máquina virtual de Java (JVM).

La Figura 1 muestra la interfaz web del NameNode de HDFS (puerto 9870) abierta en el navegador. El título "Overview 'namenode:9000' (active)" confirma que el nodo maestro está activo. La página informa la versión de Hadoop (3.2.1), la hora de inicio del servicio, el identificador del clúster y, en el resumen, que la seguridad y el modo seguro (Safemode) están desactivados y que la capacidad configurada es de 1006.85 GB.

Figura 1

Interfaz web del NameNode: resumen del clúster

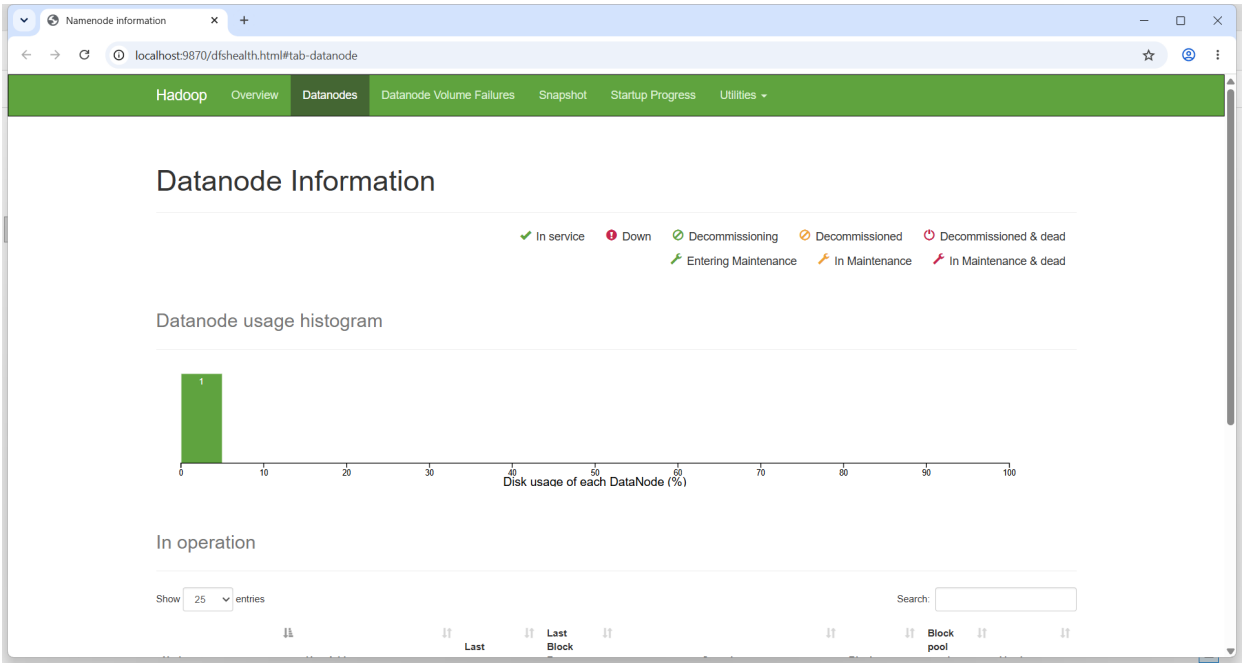


Nota. Elaboración propia. Captura de pantalla del equipo de trabajo.

La Figura 2 muestra la pestaña Datanodes. El histograma de uso de disco tiene una sola barra, con un nodo y un uso cercano a 0 %: el clúster tiene un único datanode.

Figura 2

Pestaña Datanodes del NameNode

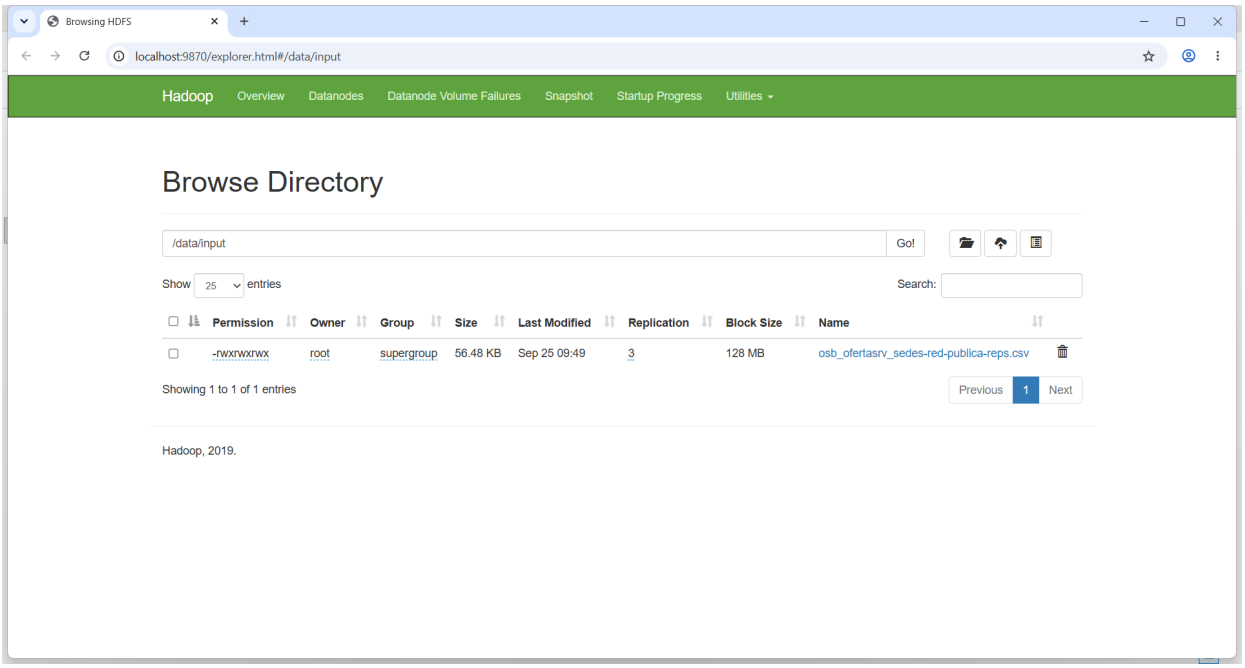


Nota. Elaboración propia. Captura de pantalla del equipo de trabajo.

La Figura 3 muestra el explorador de archivos de HDFS en la ruta /data/input. Allí está osb_ofertasrv_sedes-red-publica-reps.csv, de 56.48 KB (57 837 bytes), con permisos rwxrwxrwx, factor de replicación 3 y tamaño de bloque de 128 MB. Es el archivo que se cargó en la actividad anterior y que lee el job; como pesa mucho menos que un bloque, ocupa un solo split.

Figura 3

Archivo de entrada en HDFS (/data/input)



Nota. Elaboración propia. Captura de pantalla del equipo de trabajo.

3.3 Diseño de los Jobs MapReduce

Se implementó una sola clase parametrizable, ConteoSedes, que se ejecuta cuatro veces cambiando la clave de agrupación. El mapper emite por cada sede el par (clave, "1,e"); el combiner y el reducer suman ambos componentes, de modo que la salida de cada línea es: clave, tabulador, total de sedes, coma, sedes con atención 24 horas. Una sede se considera de atención 24 horas si en cualquiera de los siete campos de horario aparece una expresión como "00 a 24", "24 horas", "24h" o "00:00 - 24:00". Es la misma regla que aplica el job de Spark de la actividad anterior, así que los resultados son comparables.

Tabla 3

Análisis ejecutados con el job ConteoSedes

Modo	Clave de agrupación	Pregunta analítica
localidad	Código y nombre de la localidad	¿Cómo se distribuye la oferta entre las localidades?
subred	Subred integrada de servicios de salud	¿Qué subred concentra más sedes y qué parte es de atención 24 h?

Modo	Clave de agrupación	Pregunta analítica
zona	Zona urbana o rural	¿Qué proporción de la oferta se ubica en zona rural?
nivel	Nivel de complejidad	Verificación de calidad del dato: ¿el campo discrimina?

Nota. Elaboración propia.

4. Implementación

4.1 Estructura del Repositorio

Todo el código, los datos, los resultados y los registros de ejecución se organizaron en un repositorio de GitHub (sección 8), con la estructura de la Tabla 4.

Tabla 4

Estructura del repositorio

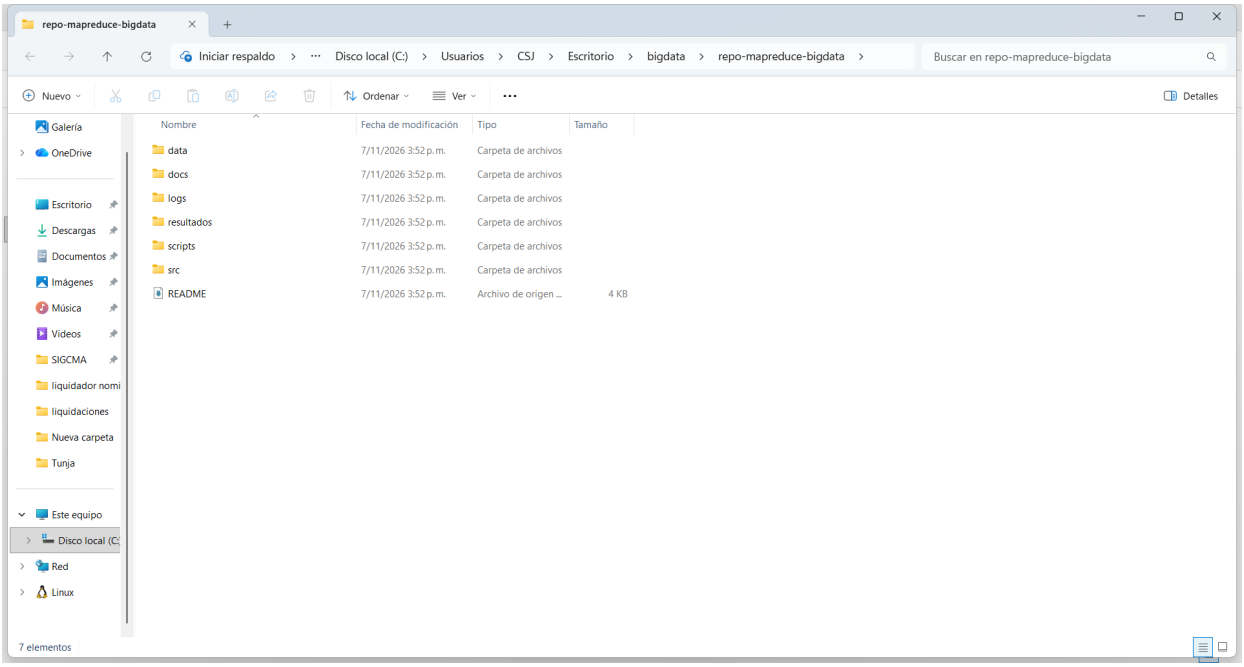
Ruta	Contenido
src/ConteoSedes.java	Mapper, Combiner/Reducer y Driver del job MapReduce.
scripts/ejecutar_mapreduce.sh	Compila, empaqueta y ejecuta los cuatro análisis dentro del contenedor namenode.
scripts/generar_figuras.py	Genera las gráficas de resultados/figuras/ con matplotlib.
data/	Archivo CSV de entrada del REPS.
resultados/	Salida de cada job (resultado_*.txt) y figuras generadas (figuras/).
logs/	Registro completo de cada job, con los contadores de Hadoop.
docs/	Informe técnico (PDF y DOCX) y capturas de pantalla (docs/evidencias/).
README.md	Descripción del proyecto y pasos para reproducirlo.

Nota. Elaboración propia.

La Figura 4 muestra la carpeta repo-mapreduce-bigdata en el Explorador de archivos de Windows, con las carpetas data, docs, logs, resultados, scripts y src y el archivo README, que corresponden a la estructura de la Tabla 4.

Figura 4

Carpeta del repositorio en el Explorador de archivos



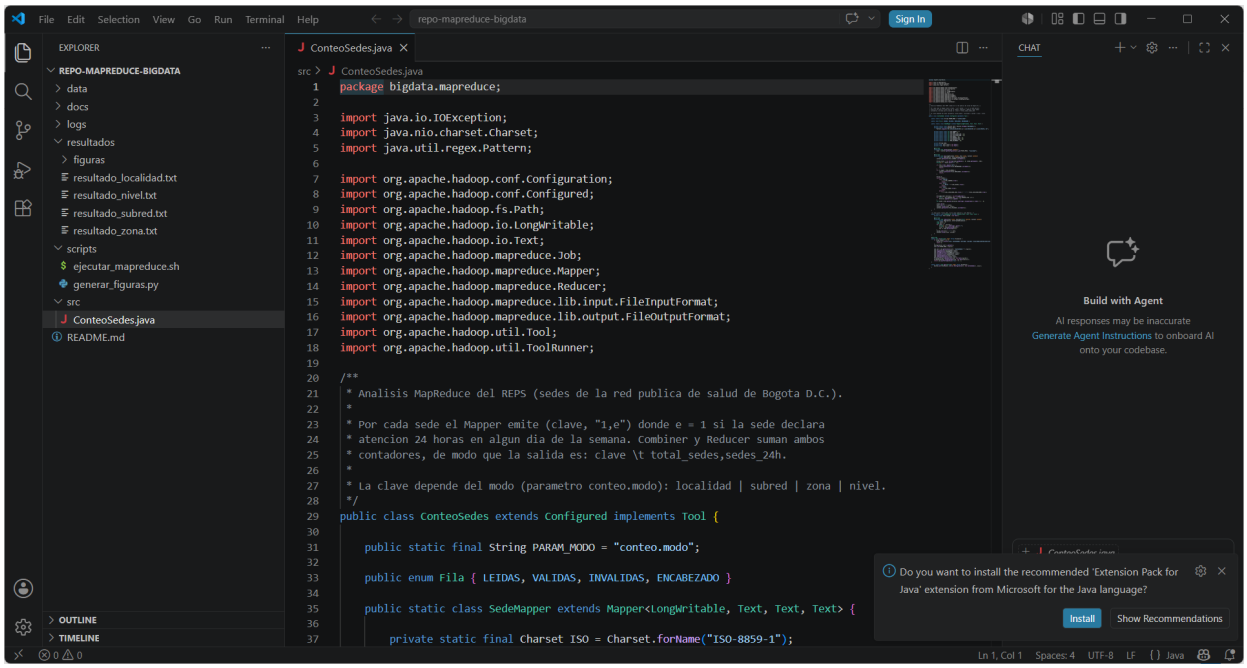
Nota. Elaboración propia. Captura de pantalla del equipo de trabajo.

4.2 Código Fuente del Job

La Figura 5 muestra el archivo src/ConteoSedes.java abierto en Visual Studio Code. A la izquierda se ve la estructura del repositorio y, en el editor, la declaración del package bigdata.mapreduce, las importaciones de la interfaz de programación (API) org.apache.hadoop.mapreduce y el comentario inicial que describe el job. El código completo está en el Anexo A.

Figura 5

Código del job ConteoSedes en Visual Studio Code




```java
package bigdata.mapreduce;

import java.io.IOException;
import java.nio.charset.Charset;
import java.util.regex.Pattern;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.conf.Configured;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.util.Tool;
import org.apache.hadoop.util.ToolRunner;

/**
 * Analisis MapReduce del REPS (sedes de la red publica de salud de Bogota D.C.).
 *
 * Por cada sede el Mapper emite (clave, "1,e") donde e = 1 si la sede declara
 * atencion 24 horas en algun dia de la semana. Combiner y Reducer suman ambos
 * contadores, de modo que la salida es: clave \t total_sedes,sedes_24h.
 *
 * La clave depende del modo (parametro conteo.modo): localidad | subred | zona | nivel.
 */
public class ConteoSedes extends Configured implements Tool {

    public static final String PARAM_MODO = "conteo.modo";

    public enum Fila { LEIDAS, VALIDAS, INVALIDAS, ENCABEZADO }

    public static class SedeMapper extends Mapper<LongWritable, Text, Text, Text> {

        private static final Charset ISO = Charset.forName("ISO-8859-1");
```

Nota. Elaboración propia. Captura de pantalla del equipo de trabajo.

El job usa la API org.apache.hadoop.mapreduce, la misma del tutorial de Hadoop (Apache Software Foundation, 2023). El mapper decodifica cada línea como ISO-8859-1, porque el archivo no está en UTF-8 y Hadoop asume UTF-8 en Text. El mismo reducer se usa como combiner, lo cual es válido porque la suma es asociativa y conmutativa y los tipos de entrada y salida del reducer coinciden. El enumerado Fila define contadores personalizados que registran cuántas líneas se leyeron, cuántas fueron válidas y cuántas se descartaron.

4.3 Script de Compilación y Ejecución

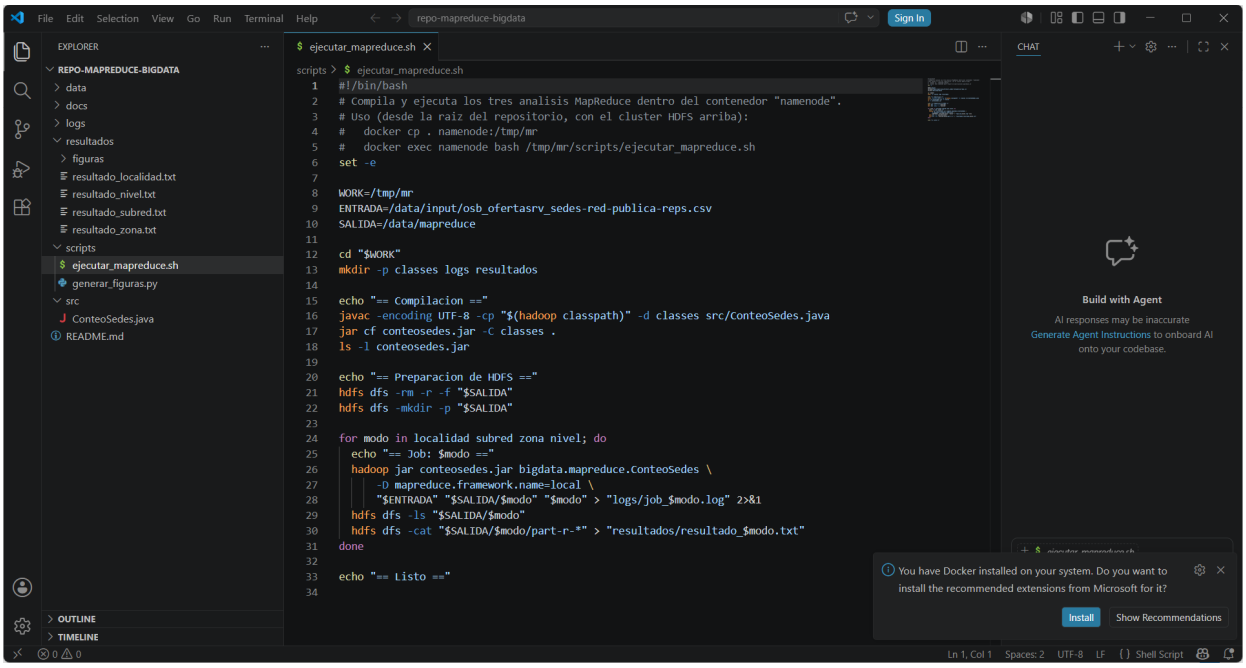
El script ejecutar_mapreduce.sh compila la clase, genera el JAR y ejecuta el job cuatro veces, una por modo. De cada ejecución guarda el log y exporta la salida de HDFS a un archivo de texto.

La Figura 6 muestra el script scripts/ejecutar_mapreduce.sh. Se distinguen sus pasos: la compilación con javac usando el classpath de Hadoop, el empaquetado en un JAR, la limpieza del directorio de salida en HDFS (hdfs dfs -rm y -mkdir) y el ciclo for que ejecuta hadoop jar una

vez por cada modo (localidad, subred, zona y nivel), con el parámetro -D mapreduce.framework.name=local. Al final de cada vuelta lista la salida en HDFS (-ls) y la guarda en un archivo de texto (-cat). El script completo está en el Anexo B.

Figura 6

Script ejecutar_mapreduce.sh en Visual Studio Code



Nota. Elaboración propia. Captura de pantalla del equipo de trabajo.

Los comandos para levantar el clúster, copiar el proyecto al contenedor, ejecutarlo y recuperar los resultados se muestran en la Figura 7.

Figura 7

Comandos utilizados para ejecutar el análisis

```
docker-compose up -d namenode datanode
docker exec -t namenode rm -rf /tmp/mr
docker cp . namenode:/tmp/mr
docker exec -t namenode bash /tmp/mr/scripts/ejecutar_mapreduce.sh
docker cp namenode:/tmp/mr/resultados/. ./resultados/
docker cp namenode:/tmp/mr/logs/. ./logs/
```

Nota. Elaboración propia. Comandos ejecutados desde la carpeta del repositorio.

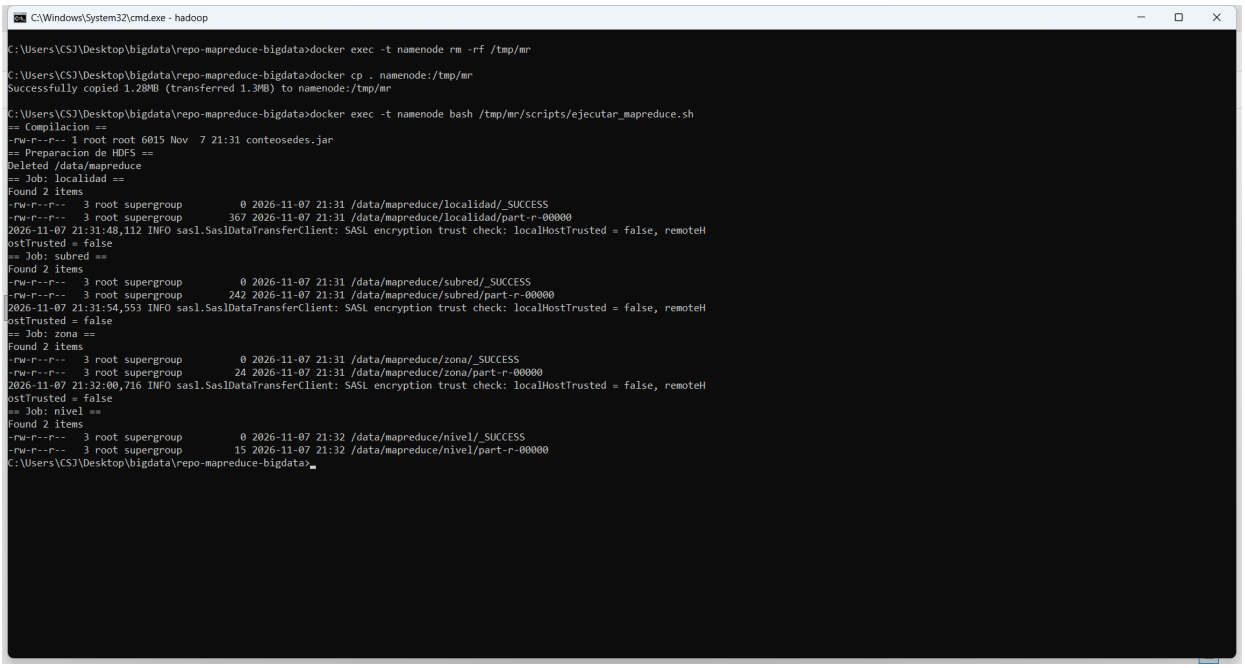
4.4 Ejecución del Procesamiento

La Figura 8 muestra la terminal (símbolo del sistema) con los tres comandos que lanzan el análisis. El primero, docker exec -t namenode rm -rf /tmp/mr, limpia la carpeta de trabajo del

contenedor; el segundo, docker cp . namenode:/tmp/mr, copia el repositorio (1.28 MB) al contenedor; y el tercero ejecuta ejecutar_mapreduce.sh. La salida del script muestra la compilación (conteosedes.jar, de 6 015 bytes), el borrado del directorio /data/mapreduce en HDFS y los cuatro jobs en orden: localidad, subred, zona y nivel. Cada uno deja en HDFS un archivo _SUCCESS y un part-r-00000 de 367, 242, 24 y 15 bytes, respectivamente. Las líneas INFO de sasl.SaslDataTransferClient son mensajes informativos del cliente de HDFS.

Figura 8

Terminal: ejecución de los cuatro jobs

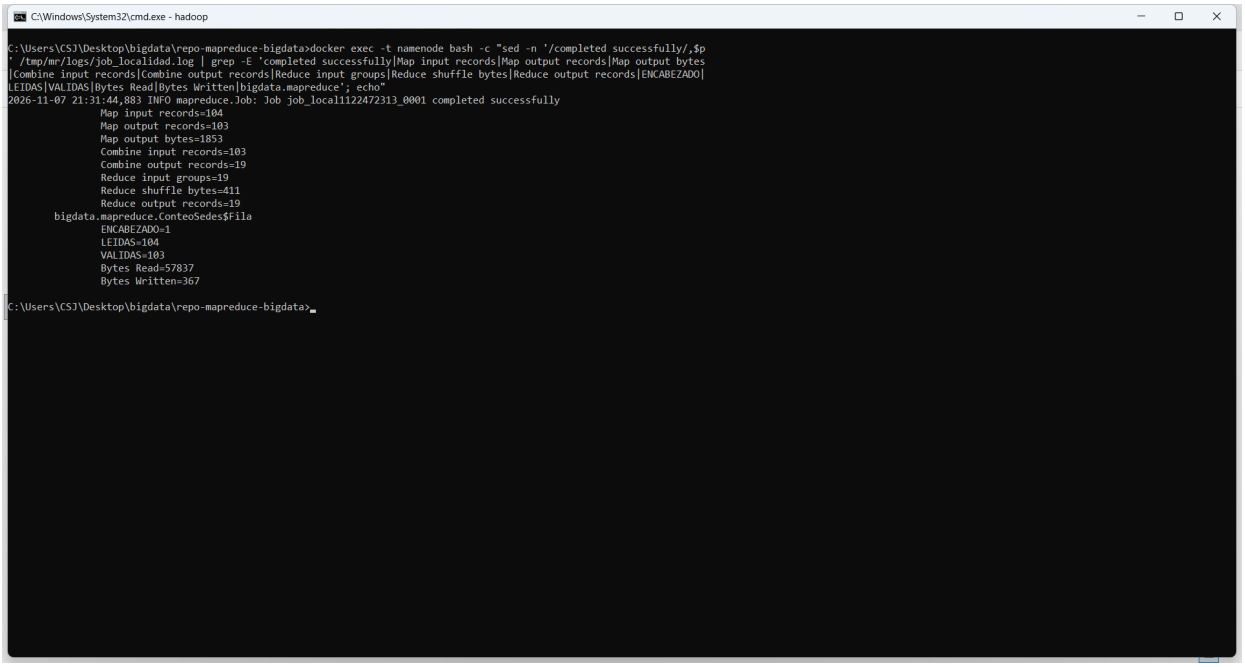


Nota. Elaboración propia. Captura de pantalla del equipo de trabajo.

La Figura 9 muestra, también en la terminal, los contadores que Hadoop registró al terminar el job de localidades (extraídos del log con sed y grep). Se lee "completed successfully" y, debajo, los valores de la Tabla 5: 104 registros de entrada, 103 emitidos por el mapper, 103 entradas al combiner y 19 salidas, 19 grupos en el reducer, 411 bytes en el shuffle y 19 registros de salida. Los contadores personalizados del job (bigdata.mapreduce.ConteoSedes$Fila) indican 1 encabezado, 104 líneas leídas y 103 válidas. Al final aparecen los bytes leídos (57 837) y escritos (367).

Figura 9

Terminal: contadores del job en modo localidad



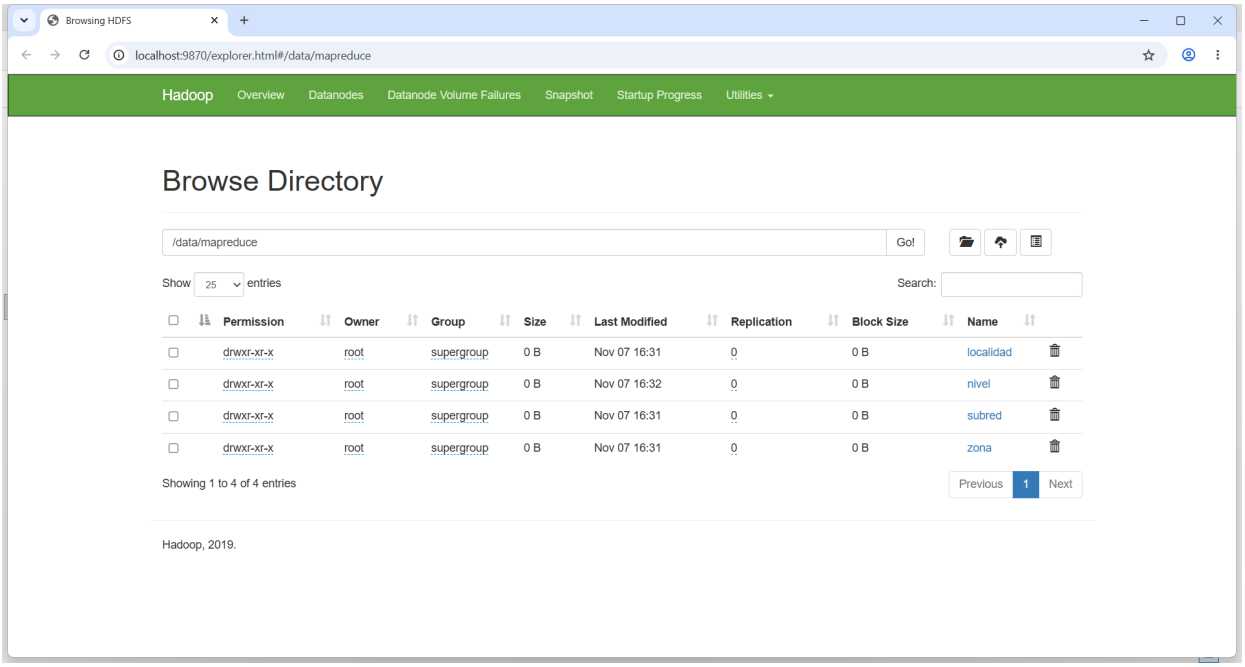
Nota. Elaboración propia. Captura de pantalla del equipo de trabajo.

Los resultados quedan en HDFS, y la interfaz web del NameNode permite revisarlos sin usar la línea de comandos.

La Figura 10 muestra el directorio /data/mapreduce en el explorador de HDFS. Contiene cuatro subdirectorios, uno por modo del job (localidad, nivel, subred y zona), todos con permisos drwxr-xr-x y propiedad del usuario root.

Figura 10

Directorio /data/mapreduce en la interfaz web de HDFS

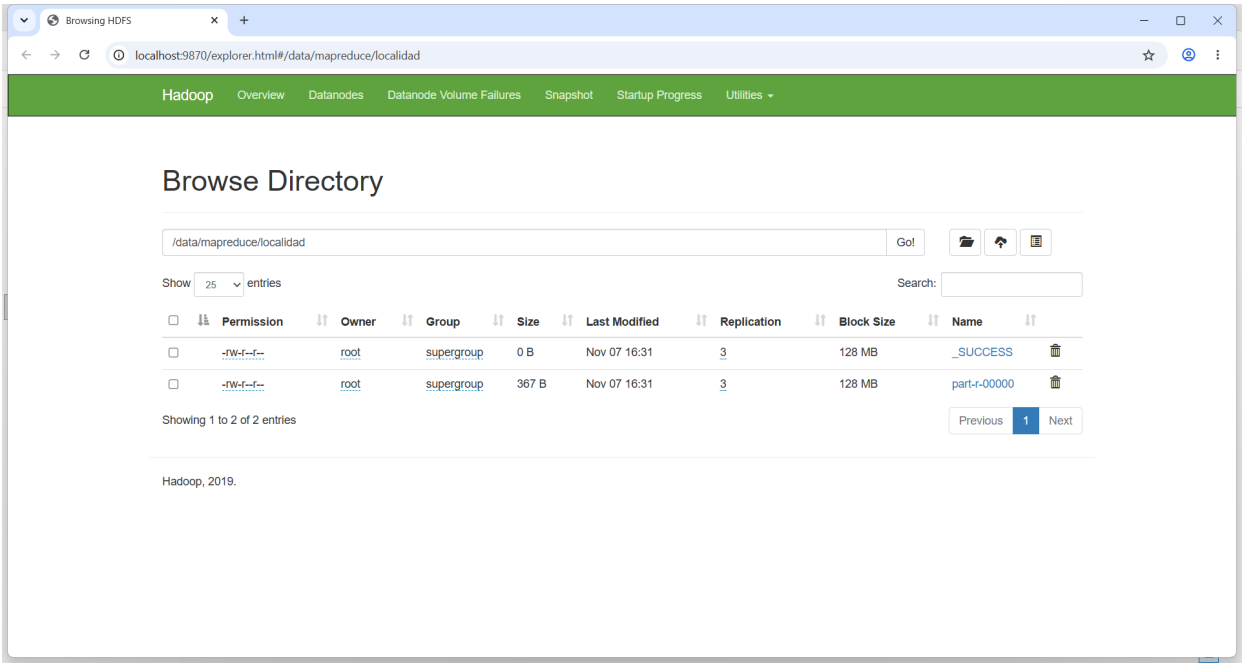


Nota. Elaboración propia. Captura de pantalla del equipo de trabajo.

La Figura 11 muestra el contenido del directorio de salida del job por localidad: el archivo _SUCCESS, de 0 bytes, que Hadoop crea cuando el job termina bien, y part-r-00000, de 367 bytes, que contiene el resultado. Ese tamaño coincide con el contador Bytes Written del job. Ambos archivos tienen factor de replicación 3 y bloque de 128 MB.

Figura 11

Contenido de /data/mapreduce/localidad



Nota. Elaboración propia. Captura de pantalla del equipo de trabajo.

5. Resultados

5.1 Contadores de Ejecución

El job leyó 104 líneas: 1 de encabezado y 103 de datos, todas válidas, de modo que no se descartó ningún registro (el contador INVALIDAS no aparece porque nunca se incrementó). El mapper emitió 103 pares y el combiner los redujo a 19 antes del shuffle (81.6 % menos registros), y el volumen transferido al reducer bajó de 1 853 a 411 bytes (77.8 % menos). Es la ventaja que describe el tutorial para el combiner: agregar localmente antes de mover los datos (Apache Software Foundation, 2023).

Tabla 5

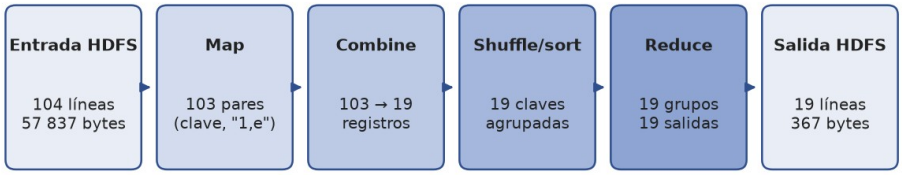
Contadores del job ConteoSedes en modo localidad

Contador	Valor	Interpretación
Map input records	104	Líneas leídas del CSV (1 encabezado + 103 sedes).
Fila.ENCABEZADO	1	Línea de encabezado, omitida.
Fila.VALIDAS	103	Sedes procesadas correctamente.
Map output records	103	Pares (clave, "1,e") emitidos por el mapper.
Map output bytes	1 853	Tamaño de la salida del mapper antes del combiner.
Combine input records	103	Pares recibidos por el combiner.
Combine output records	19	Pares tras la agregación local (uno por localidad).
Reduce shuffle bytes	411	Bytes transferidos al reducer.
Reduce input groups	19	Claves distintas (localidades).
Reduce output records	19	Líneas del resultado final.
Bytes leídos / escritos (File Format Counters)	57 837 / 367	Entrada y salida del job.

Nota. Elaboración propia a partir del registro logs/job_localidad.log.

Figura 12

Flujo del job con los contadores reportados por Hadoop



Nota. Elaboración propia en Python (matplotlib) a partir de logs/job_localidad.log.

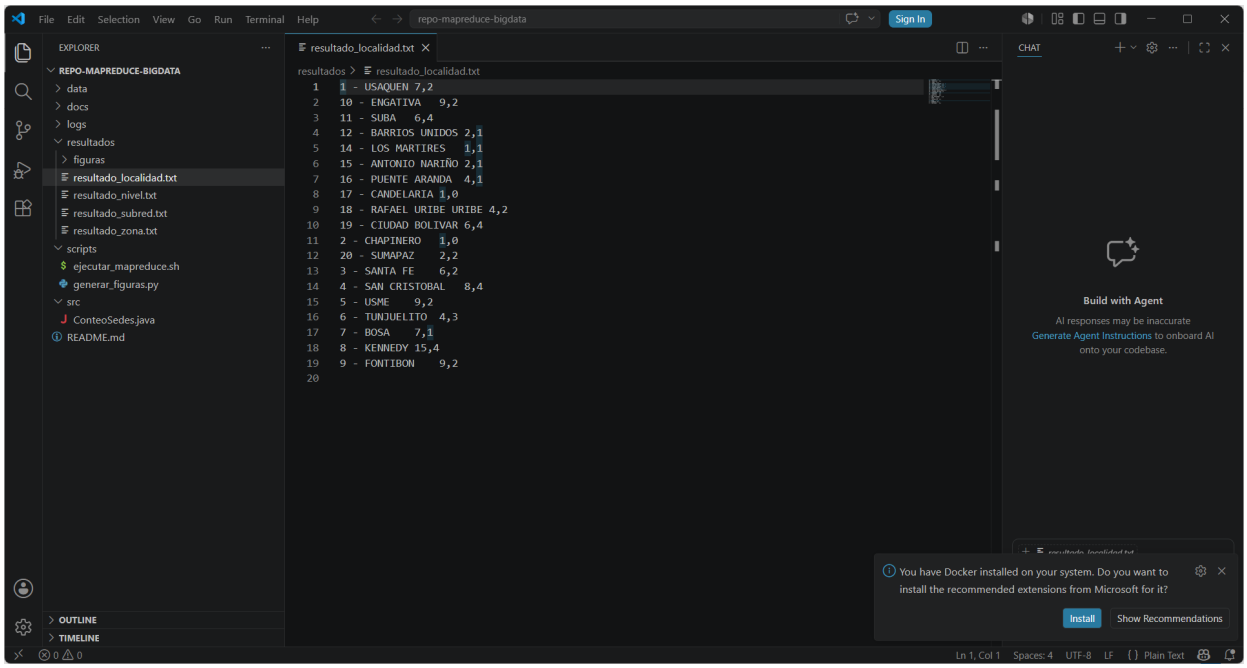
5.2 Sedes por Localidad

La oferta pública suma 103 sedes en 19 localidades. Kennedy concentra 15 (14.6 %), seguida de Fontibón, Usme y Engativá con 9 cada una; las cinco primeras localidades reúnen 50 sedes (48.5 %). De las 103 sedes, 38 (36.9 %) declaran atención las 24 horas. Chapinero y La Candelaria cuentan con una sola sede y ninguna es de atención permanente.

La Figura 13 muestra en Visual Studio Code el archivo resultados/resultado_localidad.txt, copia de la salida que el job dejó en HDFS. Cada línea tiene el código y el nombre de la localidad, un tabulador, el total de sedes y las sedes con atención 24 horas; por ejemplo, la línea "8 - KENNEDY 15,4" indica 15 sedes, 4 de ellas de atención permanente. La Tabla 6 reúne estos mismos datos ordenados por número de sedes.

Figura 13

Resultado del job en modo localidad (resultado_localidad.txt)



Nota. Elaboración propia. Captura de pantalla del equipo de trabajo.

Tabla 6

Sedes de la red pública y sedes con atención 24 horas, por localidad

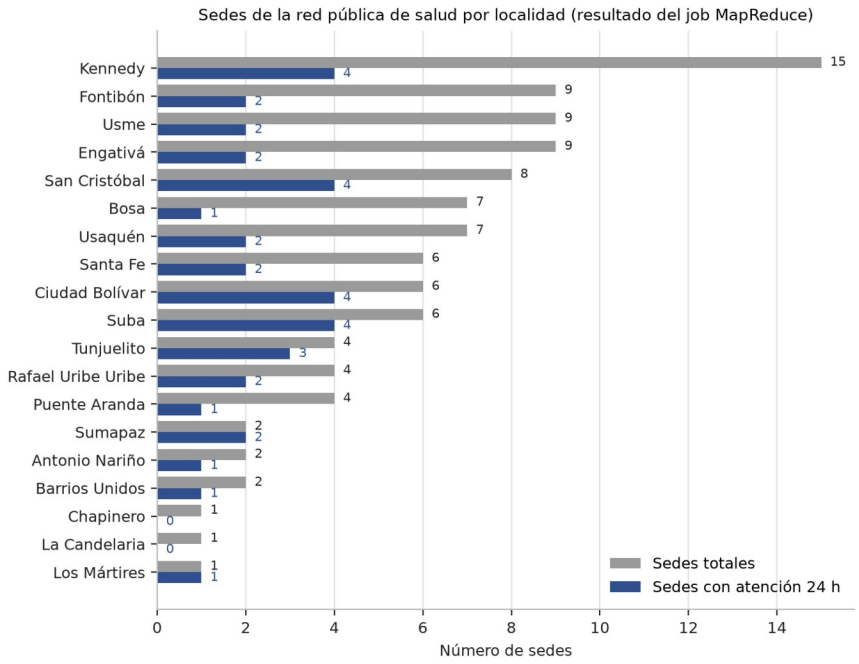
Localidad	Sedes totales	Sedes 24 h	% con atención 24 h
8 - Kennedy	15	4	26.7
5 - Usme	9	2	22.2
9 - Fontibón	9	2	22.2
10 - Engativá	9	2	22.2
4 - San Cristóbal	8	4	50.0
1 - Usaquén	7	2	28.6
7 - Bosa	7	1	14.3
3 - Santa Fe	6	2	33.3
11 - Suba	6	4	66.7
19 - Ciudad Bolívar	6	4	66.7
6 - Tunjuelito	4	3	75.0
16 - Puente Aranda	4	1	25.0
18 - Rafael Uribe Uribe	4	2	50.0
12 - Barrios Unidos	2	1	50.0
15 - Antonio Nariño	2	1	50.0
20 - Sumapaz	2	2	100.0

Localidad	Sedes totales	Sedes 24 h	% con atención 24 h
2 - Chapinero	1	0	0.0
14 - Los Mártires	1	1	100.0
17 - La Candelaria	1	0	0.0
Total	103	38	36.9

Nota. Elaboración propia a partir de resultados/resultado_localidad.txt (salida del job).

Figura 14

Sedes totales y con atención 24 horas, por localidad



Nota. Elaboración propia en Python (matplotlib) a partir de resultados/resultado_localidad.txt.

5.3 Sedes por Subred

La red está organizada en cuatro subredes integradas. Sur Occidente es la mayor, con 35 sedes (34.0 % del total), pero solo 8 de ellas (22.9 %) atienden las 24 horas; en el otro extremo, Centro Oriente tiene 50.0 %. Según el archivo, sus 35 sedes están en cuatro localidades: Kennedy, Bosa, Fontibón y Puente Aranda.

Tabla 7

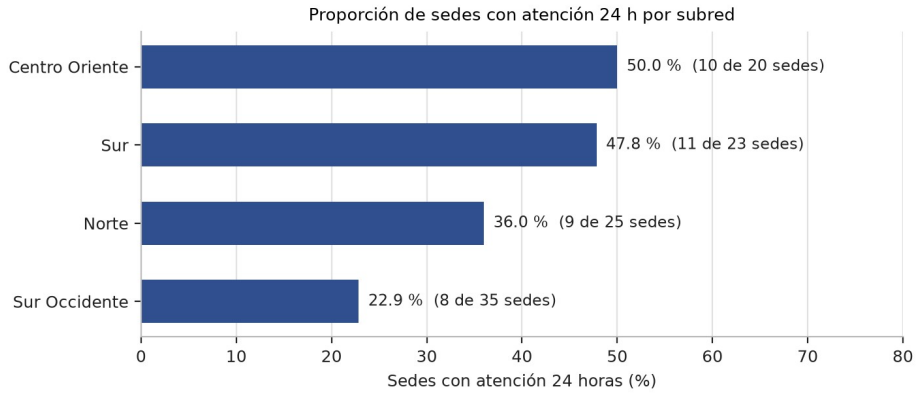
Sedes por subred integrada de servicios de salud

Subred	Sedes totales	Sedes 24 h	% con atención 24 h
Sur Occidente	35	8	22.9
Norte	25	9	36.0
Sur	23	11	47.8
Centro Oriente	20	10	50.0
Total	103	38	36.9

Nota. Elaboración propia a partir de resultados/resultado_subred.txt (salida del job).

Figura 15

Proporción de sedes con atención 24 horas, por subred



Nota. Elaboración propia en Python (matplotlib) a partir de resultados/resultado_subred.txt.

5.4 Distribución por Zona y Verificación del Nivel

La oferta es predominantemente urbana: 100 sedes son urbanas y 3 rurales. De las rurales, 2 atienden las 24 horas (66.7 %), frente a 36.0 % en zona urbana. El análisis por nivel de complejidad arrojó un único valor: las 103 sedes están en el nivel 3. Es un hallazgo de calidad de datos: el campo es constante en este conjunto y no sirve para diferenciar sedes. Conviene verificar con la fuente si el valor es correcto o si el campo no se diligencia por sede.

Tabla 8

Sedes por zona y por nivel de complejidad

Agrupación	Valor	Sedes totales	Sedes 24 h
Zona	Urbana	100	36
Zona	Rural	3	2
Nivel de complejidad	3	103	38

Nota. Elaboración propia a partir de resultados/resultado_zona.txt y resultados/resultado_nivel.txt.

5.5 Validación Cruzada con Apache Spark

Para comprobar la corrección del job se compararon sus resultados por localidad con los que produjo el job Consulta.py de Spark (Apache Spark, 2018) en la actividad anterior sobre el mismo archivo. Spark agrupó por el campo Localidad, es decir, por el código numérico. Las 19 localidades coinciden en el total de sedes y en las sedes con atención 24 horas (Tabla 9). Esto indica que ambos motores aplican la misma regla y que la lectura del CSV en MapReduce es correcta, incluida la decodificación de caracteres.

Tabla 9

Comparación de los resultados de MapReduce y Spark por localidad

Código de localidad	MapReduce: total	MapReduce: 24 h	Spark: total	Spark: 24 h	¿Coincide?
8	15	4	15	4	Sí
5	9	2	9	2	Sí
9	9	2	9	2	Sí
10	9	2	9	2	Sí
4	8	4	8	4	Sí
1	7	2	7	2	Sí
7	7	1	7	1	Sí
3	6	2	6	2	Sí
11	6	4	6	4	Sí
19	6	4	6	4	Sí
6	4	3	4	3	Sí
16	4	1	4	1	Sí
18	4	2	4	2	Sí
12	2	1	2	1	Sí
15	2	1	2	1	Sí
20	2	2	2	2	Sí

Código de localidad	MapReduce: total	MapReduce: 24 h	Spark: total	Spark: 24 h	¿Coincide?
2	1	0	1	0	Sí
14	1	1	1	1	Sí
17	1	0	1	0	Sí

Nota. Elaboración propia a partir de resultados/resultado_localidad.txt y de la salida del job Consulta.py (Spark) de la actividad anterior.

6. Discusión

6.1 Interpretación para la Toma de Decisiones

1. La oferta está concentrada: cinco localidades reúnen 48.5 % de las sedes. Kennedy, con 15 sedes, es la de mayor oferta, pero solo 26.7 % de sus sedes atiende las 24 horas; el mismo patrón se da en Bosa (1 de 7, 14.3 %).
2. Chapinero y La Candelaria no tienen ninguna sede de atención las 24 horas dentro de la red pública. Antes de concluir que hay un vacío de cobertura, habría que revisar si sus habitantes son atendidos por sedes de localidades vecinas o por prestadores privados, que este conjunto no incluye.
3. La subred Sur Occidente presenta la mayor oferta y la menor proporción de atención 24 horas (22.9 %), mientras que Centro Oriente y Sur superan el 47 %. Es un criterio razonable para priorizar la revisión de horarios en Sur Occidente.
4. La oferta rural es pequeña: 3 sedes, 2 de ellas con atención las 24 horas. Con tan pocos datos no es posible generalizar.
5. El campo nivel, constante en las 103 sedes, muestra que conviene revisar la calidad de los datos antes de usarlos en una decisión.

6.2 Limitaciones

El archivo pesa 57 837 bytes y ocupa un solo split, así que se ejecutó un único map task y un único reducer. El ejercicio muestra el diseño y el flujo de MapReduce, pero no su comportamiento con paralelismo real ni su rendimiento frente a Spark. Como el clúster no tiene NodeManager, los jobs usaron el ejecutor local y no se evidencia la planificación por YARN que

describe el tutorial (Apache Software Foundation, 2023). HDFS tiene un solo datanode: aunque el factor de replicación configurado es 3, cada bloque existe una sola vez y HDFS lo reporta como sub-replicado. La atención 24 horas se deduce con expresiones regulares sobre el horario declarado, de modo que refleja lo que la sede informa y puede diferir del servicio efectivo. Los datos son del corte de septiembre de 2026 y no incluyen prestadores privados.

7. Conclusiones

1. Se implementó y ejecutó un job MapReduce en Java sobre datos almacenados en HDFS, siguiendo la estructura del tutorial de Hadoop (Apache Software Foundation, 2023): Mapper, Combiner, Reducer, Driver y contadores personalizados.
2. Los contadores verifican el procesamiento: se leyeron 104 líneas, se procesaron 103 sedes sin descartar ninguna y el combiner redujo los registros del shuffle de 103 a 19.
3. Los resultados de MapReduce coinciden exactamente con los de Spark en las 19 localidades, lo que valida el job.
4. La oferta se concentra en pocas localidades y 38 de las 103 sedes (36.9 %) atienden las 24 horas. La subred Sur Occidente combina el mayor tamaño con la menor proporción de atención permanente.
5. El flujo completo, con Spark para limpiar, MapReduce para analizar y gráficas para visualizar, llevó de datos crudos a indicadores interpretables y reveló un problema de calidad en el campo nivel.
6. Para evaluar el beneficio real de MapReduce se necesita un volumen mayor y un clúster con YARN y varios nodos, algo que queda como trabajo futuro.

8. Repositorio de GitHub

El código fuente, los datos, los resultados, los registros de ejecución, las capturas de pantalla y las instrucciones para reproducir el análisis están disponibles en el repositorio de GitHub del autor: https://github.com/Danielleon172025/repo-mapreduce-bigdata.

9. Referencias

Apache Hadoop. (2026). *Apache Hadoop*. Apache Software Foundation. https://hadoop.apache.org/

Apache Software Foundation. (15 de marzo de 2023). *MapReduce tutorial* (Versión 3.3.5). Apache Hadoop. https://hadoop.apache.org/docs/stable/hadoop-mapreduce-client/hadoop-mapreduce-client-core/MapReduceTutorial.html

Apache Spark. (2018). *Apache Spark*. Apache Software Foundation. https://spark.apache.org

Docker Inc. (2026). *Docker Desktop* (Versión 4.74.0) [Software]. https://www.docker.com/products/docker-desktop/

Holdsworth, J., & Kosinski, M. (s.f.). *¿Qué es MapReduce?* IBM. https://www.ibm.com/mx-es/think/topics/mapreduce

Secretaría Distrital de Salud de Bogotá. (2026). *Oferta de instituciones prestadoras de servicio de salud de la red pública de Bogotá D. C.* [Conjunto de datos]. datos.gov.co. https://www.datos.gov.co/dataset/Oferta-Instituciones-Prestadoras-de-Servicio-de-sa/uzte-hw6a

Universidad de Cundinamarca. (s.f.). *Guía Big Data – Análisis* [Guía de curso]. Especialización en Analítica de Datos.

Anexo A. Código Fuente de ConteoSedes.java

Figura A1

ConteoSedes.java: Mapper, Combiner/Reducer y Driver

```
package bigdata.mapreduce;

import java.io.IOException;
import java.nio.charset.Charset;
import java.util.regex.Pattern;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.conf.Configured;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
import org.apache.hadoop.util.Tool;
import org.apache.hadoop.util.ToolRunner;

/**
 * Analisis MapReduce del REPS (sedes de la red publica de salud de Bogota D.C.).
 *
 * Por cada sede el Mapper emite (clave, "1,e") donde e = 1 si la sede declara
 * atencion 24 horas en algun dia de la semana. Combiner y Reducer suman ambos
 * contadores, de modo que la salida es: clave \t total_sedes,sedes_24h.
 *
 * La clave depende del modo (parametro conteo.modo): localidad | subred | zona | nivel.
 */
public class ConteoSedes extends Configured implements Tool {

    public static final String PARAM_MODO = "conteo.modo";

    public enum Fila { LEIDAS, VALIDAS, INVALIDAS, ENCABEZADO }

    public static class SedeMapper extends Mapper<LongWritable, Text, Text, Text> {

        private static final Charset ISO = Charset.forName("ISO-8859-1");
        private static final Pattern H24 =
            Pattern.compile("24\\s*hora|24h|00:00\\s*-\\s*24:00|0:00\\s*-\\s*23:59|00 a 24");

        private static final int COL_SUBRED = 4;
        private static final int COL_ZONA = 9;
        private static final int COL_LOCALIDAD_COD = 14;
        private static final int COL_LOCALIDAD_NOM = 15;
        private static final int COL_NIVEL = 31;
        private static final int COL_HORARIO_INI = 36;
        private static final int COL_HORARIO_FIN = 42;
        private static final int MIN_COLUMNAS = 43;

        private String modo;
        private final Text clave = new Text();
        private final Text valor = new Text();

        @Override
        protected void setup(Context context) {
            modo = context.getConfiguration().get(PARAM_MODO, "localidad");
        }

        @Override
        protected void map(LongWritable offset, Text linea, Context context)
                throws IOException, InterruptedException {
            context.getCounter(Fila.LEIDAS).increment(1);

            String texto = new String(linea.getBytes(), 0, linea.getLength(), ISO);
            String[] f = texto.split(";", -1);

            if (f[0].trim().equals("ID")) {
```

```
                context.getCounter(Fila.ENCABEZADO).increment(1);
                return;
            }
            if (f.length < MIN_COLUMNAS) {
                context.getCounter(Fila.INVALIDAS).increment(1);
                return;
            }

            String k;
            switch (modo) {
                case "subred":
                    k = f[COL_SUBRED].trim();
                    break;
                case "nivel":
                    k = "Nivel " + f[COL_NIVEL].trim();
                    break;
                case "zona":
                    k = f[COL_ZONA].trim();
                    break;
                default:
                    k = f[COL_LOCALIDAD_COD].trim() + " - " + f[COL_LOCALIDAD_NOM].trim();
            }

            StringBuilder horarios = new StringBuilder();
            for (int i = COL_HORARIO_INI; i <= COL_HORARIO_FIN; i++) {
                horarios.append(f[i]).append(' ');
            }
            int es24h = H24.matcher(horarios.toString().toLowerCase()).find() ? 1 : 0;

            clave.set(k);
            valor.set("1," + es24h);
            context.write(clave, valor);
            context.getCounter(Fila.VALIDAS).increment(1);
        }
    }

    /** Suma pares "total,24h". Sirve como Combiner y como Reducer. */
    public static class SumaReducer extends Reducer<Text, Text, Text, Text> {
        private final Text salida = new Text();

        @Override
        protected void reduce(Text clave, Iterable<Text> valores, Context context)
                throws IOException, InterruptedException {
            long total = 0;
            long h24 = 0;
            for (Text v : valores) {
                String[] p = v.toString().split(",");
                total += Long.parseLong(p[0]);
                h24 += Long.parseLong(p[1]);
            }
            salida.set(total + "," + h24);
            context.write(clave, salida);
        }
    }

    @Override
    public int run(String[] args) throws Exception {
        if (args.length != 3) {
            System.err.println("Uso: ConteoSedes <entrada> <salida> <localidad|subred|zona|nivel>");
            return 2;
        }
        Configuration conf = getConf();
        conf.set(PARAM_MODO, args[2]);

        Job job = Job.getInstance(conf, "ConteoSedes-" + args[2]);
        job.setJarByClass(ConteoSedes.class);
        job.setMapperClass(SedeMapper.class);
        job.setCombinerClass(SumaReducer.class);
        job.setReducerClass(SumaReducer.class);
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(Text.class);
        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));
        return job.waitForCompletion(true) ? 0 : 1;
    }

    public static void main(String[] args) throws Exception {
        System.exit(ToolRunner.run(new Configuration(), new ConteoSedes(), args));
    }
}
```

Nota. Elaboración propia. Archivo src/ConteoSedes.java del repositorio.

Anexo B. Script ejecutar_mapreduce.sh

Figura A2

ejecutar_mapreduce.sh: compilación y ejecución de los jobs

```
#!/bin/bash
# Compila y ejecuta los tres analisis MapReduce dentro del contenedor "namenode".
# Uso (desde la raiz del repositorio, con el cluster HDFS arriba):
#   docker cp . namenode:/tmp/mr
#   docker exec namenode bash /tmp/mr/scripts/ejecutar_mapreduce.sh
set -e

WORK=/tmp/mr
ENTRADA=/data/input/osb_ofertasrv_sedes-red-publica-reps.csv
SALIDA=/data/mapreduce

cd "$WORK"
mkdir -p classes logs resultados

echo "== Compilacion =="
javac -encoding UTF-8 -cp "$(hadoop classpath)" -d classes src/ConteoSedes.java
jar cf conteosedes.jar -C classes .
ls -l conteosedes.jar

echo "== Preparacion de HDFS =="
hdfs dfs -rm -r -f "$SALIDA"
hdfs dfs -mkdir -p "$SALIDA"

for modo in localidad subred zona nivel; do
  echo "== Job: $modo =="
  hadoop jar conteosedes.jar bigdata.mapreduce.ConteoSedes \
      -D mapreduce.framework.name=local \
      "$ENTRADA" "$SALIDA/$modo" "$modo" > "logs/job_$modo.log" 2>&1
  hdfs dfs -ls "$SALIDA/$modo"
  hdfs dfs -cat "$SALIDA/$modo/part-r-*" > "resultados/resultado_$modo.txt"
done

echo "== Listo =="
```

Nota. Elaboración propia. Archivo scripts/ejecutar_mapreduce.sh del repositorio.